Fast Parallel Image Rotation Algorithm

Dhrubajyoti Mandal
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22053859@kiit.ac.in

Dr. Sujoy Dutta
Asst. Prof. & Asst. CoE
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
sdattafcs@kiit.ac.in

Sourav Kumar Parida
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22051032@kiit.ac.in

Chiranjeev Bhaya
Lead Engineer (Camera System)
Samsung R&D Institute
Bangalore, India
c.bhaya@samsung.com

Subhadeep Bhadra
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
2205336@kiit.ac.in

Chandra Mohan V
Architect (Camera System)
Samsung R&D Institute
Bangalore, India
chandra.mv@samsung.com

***Abstract*—This paper presents a novel fast image rotation algorithm that leverages CPU parallelization while maintaining high image quality. The proposed method is a single-pass algorithm, derived from the existing Double-Line Rotation (DLR) method, which reduces computational complexity and generalizes the algorithm. Initially, a baseline for the given image is calculated to determine the starting line, which defines the initial point for each vertical or horizontal line in the image to be rotated. The corresponding pixels to be mapped are identified using floating point arithmetic, and trigonometric calculations are performed only once per line. This approach ensures precise image transformation with minimal computational overhead.**
***Index terms:* Image rotation, line rotation image transform, double-line rotation (DLR), parallel image rotation.**

I. Introduction

2-D Image Rotation has a number of applications in digital photography and image processing like image matching, alignment, [1] etc. Highly accurate image rotation is essential for certain image processing tasks such as feature extraction and matching. Whilst there are many state-of-the-art image rotation algorithms, they have limitations in terms of image quality and speed [2] [3]. Parallel algorithms for image processing aim to execute faster than sequential algorithms, however designing and scheduling a fast parallel image rotation is a challenge [4]. The most straightforward, and perhaps the most well known approach to implement image rotation is by using the rotation matrix [5], as described here

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x - cx \\ y - cy \end{bmatrix} * \begin{bmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{bmatrix}$$

where, θ is the angle of rotation, x' and y' are the new coordinates of the rotated image, and (cx, cy) is the center of the original image, which is treated as the axis of rotation. Assuming the centre of rotation of the new image is (cx', cy'), the final new coordinates are:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x' + cx' \\ y' + cy' \end{bmatrix}$$

II. Literature Review

Affine transformation is a fundamental technique in computer graphics and computational geometry, extensively employed for various image manipulation operations such as translation, scaling, rotation, shearing, and reflection. Mathematically, it is represented as $y = Ax + b$ where A is a 2×2 matrix that defines the linear transformation, and b is a translation vector. This formulation ensures the preservation of geometric properties such as points, straight lines, and planes, making affine transformation indispensable for numerous graphical applications. In the context of image rotation, the matrix $\mathbf{A}$ is typically determined using trigonometric functions based on the rotation angle θ, while the translation components t_x and t_y are calculated to center the rotated image within its new dimensions. The `warpAffine` function in OpenCV is widely utilized to perform such affine transformations efficiently, offering flexibility in handling various interpolation methods like nearest-neighbor, bilinear, and bicubic. These interpolation methods are crucial for minimizing distortions and preserving image quality during the transformation [6].

However, affine transformations are not without limitations, especially when dealing with large rotation angles or combined

transformations, such as scaling and shearing, which can lead to artifacts like aliasing and loss of detail.

To mitigate these issues, more sophisticated methods have been developed. One such method is the Hermite polynomial expansion, which leverages the orthogonality of Hermite polynomials to achieve smooth and accurate interpolation. This technique is particularly effective for image processing tasks requiring high precision. The image function is expanded using two-dimensional Hermite polynomials, and rotation is performed by transforming the coordinates with a rotation matrix. The rotated image is then reconstructed using inverse Hermite expansion, ensuring smooth transitions and minimal distortion. Despite its accuracy, the Hermite expansion method is computationally intensive, requiring the evaluation and summation of multiple polynomial terms, which can result in slower processing times. Additionally, while this method excels at smooth interpolation, it may struggle to maintain fine details in images with high-frequency content [3].

Another advanced technique is the Double Line Rotation (DLR) method, which is recognized for its ability to maintain geometric and intensity integrity during transformation. The DLR method divides the image into eight distinct zones, each governed by specific rotation equations and transformations. This zonal approach ensures that each pixel in the original image is mapped to two corresponding pixels in the rotated image, maintaining a consistent distance ratio between points. This feature is crucial for applications that require feature extraction invariance, making the DLR method particularly suitable for pattern recognition and computer vision tasks. However, the complexity of managing multiple zones and their corresponding transformations can lead to inefficiencies, especially with high-resolution images. Moreover, distortions may occur at the boundaries between zones, potentially degrading the visual quality of the rotated image [2].

III. Proposed Method

The proposed method is a single pass algorithm, which means that it performs the entire image rotation operation at once. The proposed algorithm predominantly affects pixels in the row major (width), or ones in the column major (height), depending on the angle of rotation, α. If $\alpha \in \left(\frac{\pi}{4}, \frac{3\pi}{4}\right] \bigcup \left(\frac{5\pi}{4}, \frac{7\pi}{4}\right]$, rotation is performed in the ***column major***, and it is performed in the ***row major*** for the rest of the angles. Accordingly, for rotating the source image, entire row or column vectors of pixels are rotated as a single unit, instead of the more common approach, where each individual pixel is separately mapped. Due to this reason, it is necessary to accurately determine the initial coordinates from which the new row or column vector will begin. The coordinates of the rest of the pixels in the vector can then be determined by simple floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

Apart from this, depending on α, the pixel processing order changes - i.e., whether the last row or column vector of pixels get processed first, or last. Since it has been found out that these values are very sensitive to α, the specific details of which are discussed in the later parts of this paper.

A. Rotation Of An Image In The Column Major

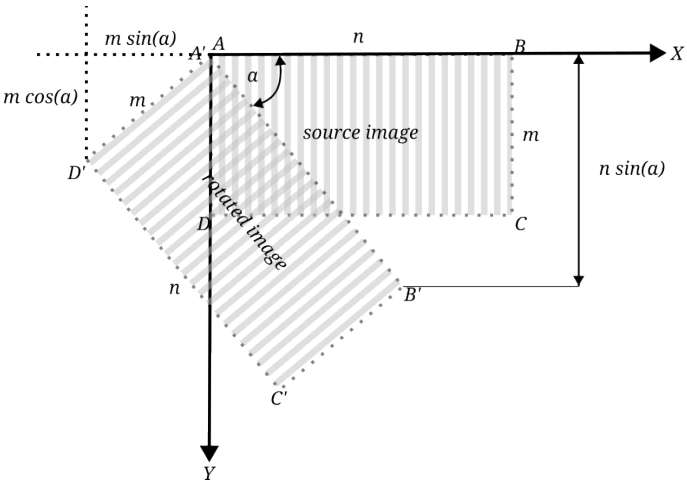

Fig. 1: Example demonstrating rotation of an image in the column major. Every column vector of pixels are rotated at once (Represented by the stripes in the image).

As seen in the example image in figure 1, for the given angle of rotation, column vectors were rotated (represented by the stripes). Each such vector to be rotated begins from the line $\overrightarrow{A'B'}$. As such, the initial coordinates of each of these vectors can be determined by the equation of the same line, which is given by:

$$f(i) = \left\lfloor \frac{i}{tan(\alpha)} \right\rfloor \forall \{i \in \mathbb{Z},\ i \in [0, n \cdot sin(\alpha))\} \qquad (1)$$

Thus, the initial coordinates of the column vectors would be of the type $(f(i), i)$. Next, to determine the coordinates of the rest of the pixels in the column vector, as mentioned previously, we perform floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

In the example in figure 1, we can see that each column vector is spread through a length of $m \cdot \sin(\alpha)$ in the x-axis and through a length of $m \cdot \cos(\alpha)$ in the y-axis. It may also be observed that these values are the projections of any line perpendicular to line defined by the function in equation 1 on the coordinate axes.

We now define:

$$\delta x = |sin(\alpha)|$$

$$\delta y = |cos(\alpha)|$$

The coordinates of the next pixel in the column vector is then given by:

$$(f(i) + \delta x, i + \delta y)$$

The coordinates of the pixel following that is determined by adding δx and δy to the previous coordinates accordingly, and

so on, for each of the m pixels in the given column vector. The end-result is that this operation covers all the $m \cdot \sin(\alpha)$ pixels in the x-axis and all the $m \cdot \cos(\alpha)$ pixels in the y-axis, therefore mapping all of the m pixels in the specific column vector.

This process continues for all of the n column vectors in the image, until the entire image is rotated. In practice, each column vector is actually mapped **twice**. The first mapping happens from the $i\,th$ column vector in I to the $i\,th$ column vector in R. The second mapping happens from the ith column vector in I to the $(i+1)\,th$ column vector in R. Each of these $(i+1)\,th$ column vectors get overwritten in each iteration with a new column vector in I; however, this gives a sufficiently accurate issusion of anti-aliasing without losing much quality.

The signs for δx and δy depend on whether, while mapping the column vectors in the rotated image, the values of the x-coordinate y-coordinate are decreasing or increasing. In the given example in figure 1, δx is negative, but δy is positive. There is a third parameter, defined as:

$$\delta i = \frac{1}{\delta x}$$

δi determines the rate at which the column vector is processed from one to the next, and its sign depends on the angle of rotation, α. The calculations for δx, δy, and δi only happen once per column vector in the image, so the entire image transformation is quite fast, overall.

B. Rotation Of An Image In The Row Major

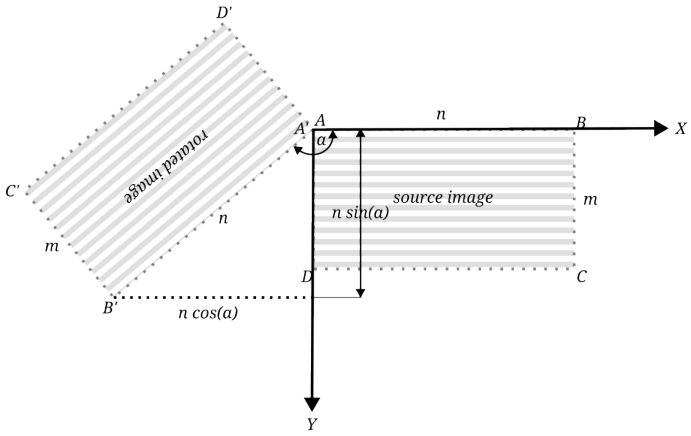

Fig. 2: Example demonstrating rotation of an image in the row major. Every row vector of pixels are rotated at once (Represented by the stripes in the image).

In the example image in figure 2, for the given angle of rotation, row vectors were rotated (represented by the stripes). All the calculations and logic is very similar to the one described for rotation in the column major [III-A]. Here, each vector to be rotated begins from the line $\overrightarrow{A'D'}$, whose equation is given by:

$$f(i) = \left\lfloor \frac{i}{tan(\alpha + \frac{\pi}{2})} \right\rfloor \forall \{i \in \mathbb{Z},\ i \in [0, m \cdot cos(\alpha))\} \quad (2)$$

TABLE I: Notation

Symbol	Description
I	Original image
m, n	Height, width of I
R	Rotated image
m', n'	Height, width of R
α	Angle of rotation in radians
x'	Offset by which the x-coordinates of R need to be shifted
y'	Offset by which the y-coordinates of R need to be shifted
$f(...)$	Function defining the equation of the line from where the row or column vector starts [eq. 1] [eq. 2]
r_o	Upper limit of the function f
N	Number of elements (pixels) in the row or column vector f
`flip`	Boolean value that changes pixel processing order based on α
`row_major`	Boolean value that determines whether processing is to be done in the row or column major
`negate`	Boolean value, that if true, negates the indices used for indexing the pixels of the image

Note: The algorithm described in [Algorithm 1] follows ***Python*** style indexing. For example, if `a = [[1, 2, 3], [5, 6]]`, then `a[0][-1]` represents the element `3`.

Everything else stays the same, except the fact that now, each row vector is spread through a length of $n \cdot \cos(\alpha)$ in the x-axis and through a length of $n \cdot \sin(\alpha)$ in the y-axis, and accordingly, we have:

$$\delta x = |cos(\alpha)|$$

$$\delta y = |sin(\alpha)|$$

C. Implementation

Algorithm 1 Image Rotation Algorithm

procedure ROTATE(...) ▷ Performs image rotation on R
  `src_i` $= 0$ ▷ Initialize internal control variables
  `S` $= 1$
  $M = [\,]$
  if `negate` **then**
    `S = -1`
  end if
  if `flip` **then**
    $M = [m_i]\ for\ i = 0, 1, \ldots, N-1,\ |\ m_i = N - i$
  else
    $M = [m_i]\ for\ i = 0, 1, \ldots, N-1,\ |\ m_i = N$
  end if
  for $i = 0$ to r_o **do**
    `temp_x` $= f(i)$
    `temp_y` $= i$
    for px in M **do**
      $x = \lfloor$ `temp_x` $\rfloor$
      $y = \lfloor$ `temp_y` $\rfloor$
      `temp_i` $= \lfloor$ `src_i` $\rfloor$
      if `row_major` **then**
        $a =$ `temp_i`
        $b = px$
      else

$a = px$
$b =$ `temp_i`
end if
▷ Map rotated pixels to R
$R[\ \mathtt{S} * (y + y'),\ \ \mathtt{S} * (x + x')] = I[a, b]$
$R[\ \mathtt{S} * (y + y') + 1,\ \ \mathtt{S} * (x + x')] = I[a, b]$
`temp_x` += δx
`temp_y` += δy
end for
`src_i` += δi
end for
end procedure

The algorithm described above [Algorithm 1] rotates image I to R.
R is Initialized as a blank image of width n' and height m' to fit R in the canvas, where:

$$m' = \lceil |m \cdot cos(\alpha) + n \cdot sin(\alpha)| \rceil$$
$$n' = \lceil |m \cdot sin(\alpha) + n \cdot cos(\alpha)| \rceil$$

The rotation function is then invoked, with each of the parameters updating depending on the value of α, as discussed above. The detailed values of each of the parameters are mentioned in [Table II].

Note: It has been observed, that for ranges of α which are vertically opposite each other (For example, angles in the range $\left(\frac{\pi}{4}, \frac{\pi}{2}\right]$ are vertically opposite angles in the range $\left(\frac{5\pi}{4}, \frac{3\pi}{2}\right]$, and so on), the only parameters of rotation that change are δi, `negate`, and `flip`. As such, the calculations have been generalized for such angles and they have been represented in [Table II], with a ($'$) at the end of their names (shaded cells).

Algorithm 2 Driver Function

procedure MAIN(...) ▷ Invokes [Algorithm 1]
Determine the boolean value of `row_major`
Determine ***f***, based on the value of `row_major`
Initialize a blank image R with dimensions (m', n')
Determine the parameters of rotation from [Table II]
Generate R using [Algorithm 1]
end procedure

IV. Experimental Results and Analysis

In this section, we perform an accuracy analysis of the reconstructed images using various error metrics. These metrics quantify the similarity between the original and the processed images, providing insight into the effectiveness of the image processing techniques. The following subsections describe each metric in detail, including their mathematical formulation and interpretation.

A. Peak Signal-to-Noise Ratio (PSNR)

PSNR is a widely used metric for evaluating the quality of reconstructed or compressed images. It quantifies the ratio between the maximum possible pixel value of the image and the mean squared error (MSE) between the original and the processed images. PSNR is expressed in decibels (dB), with higher values indicating better image quality.

$$\text{PSNR} = 20 \cdot \log_{10}\left(\frac{\text{MAX}_I}{\sqrt{\text{MSE}}}\right) \tag{3}$$

The Mean Squared Error (MSE) is calculated as:

$$\text{MSE} = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}[I(i,j) - K(i,j)]^2 \tag{4}$$

where MAX_I represents the maximum possible pixel value (e.g., 255 for 8-bit images), and $I(i,j)$ and $K(i,j)$ are the pixel values at position (i,j) in the original and processed images, respectively.

B. Correlation Coefficient (CC)

The Correlation Coefficient (CC) is a statistical measure that quantifies the similarity between two images by assessing the correlation between corresponding pixels. CC ranges from -1 to 1, with 1 indicating perfect positive correlation, -1 indicating perfect negative correlation, and 0 indicating no correlation.

$$\text{CC} = \frac{\sum_{i=1}^{M}\sum_{j=1}^{N}\left(I(i,j) - \overline{I}\right)\left(K(i,j) - \overline{K}\right)}{\sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N}\left(I(i,j) - \overline{I}\right)^2 \cdot \sum_{i=1}^{M}\sum_{j=1}^{N}\left(K(i,j) - \overline{K}\right)^2}} \tag{5}$$

Here, $\overline{I}$ and $\overline{K}$ represent the mean pixel values of the original and processed images, respectively.

C. Laplacian Mean Squared Error (LMSE)

Laplacian Mean Squared Error (LMSE) focuses on the high-frequency components of an image, particularly edges. By applying a Laplacian filter to both the original and processed images, LMSE calculates the error in edge details, which is particularly useful for detecting blurring or loss of fine details.

$$\text{LMSE} = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}[\text{Laplacian}(I(i,j)) - \text{Laplacian}(K(i,j))]^2 \tag{6}$$

Lower LMSE values indicate less error in edge preservation, thus implying higher image quality.

TABLE II: Rotation Parameters, based on α

Range of α	Rotation parameters											
	x'	y'	δx	δy	δi	r_o	**N**	`negate`	`flip`	$\delta i'$	`negate'`	`flip'`
$[0, \frac{\pi}{4}]$	$\lceil \lvert m \cdot \sin(\alpha) \rvert \rceil$	0	$\lvert \cos(\alpha) \rvert$	$\lvert \sin(\alpha) \rvert$	$1/\delta x$	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	n	`false`	`false`	$1/\delta x$	`true`	`false`
$(\frac{\pi}{4}, \frac{\pi}{2}]$	$\lceil \lvert m \cdot \sin(\alpha) \rvert \rceil$	0	$-\lvert \sin(\alpha) \rvert$	$\lvert \cos(\alpha) \rvert$	$-1/\delta x$	$\lceil \lvert n \cdot \sin(\alpha) \rvert \rceil$	m	`false`	`false`	$1/\delta x$	`false`	`true`
$(\frac{\pi}{2}, \frac{3\pi}{4}]$	m'	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	$-\lvert \sin(\alpha) \rvert$	$-\lvert \cos(\alpha) \rvert$	$-1/\delta x$	$\lceil \lvert n \cdot \sin(\alpha) \rvert \rceil$	m	`false`	`false`	$1/\delta x$	`false`	`true`
$(\frac{3\pi}{4}, \pi]$	$\lceil \lvert n \cdot \cos(\alpha) \rvert \rceil$	0	$-\lvert \cos(\alpha) \rvert$	$\lvert \sin(\alpha) \rvert$	$1/\delta x$	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	n	`false`	`false`	$1/\delta x$	`true`	`false`

D. Relative Error (Re)

Relative Error (Re) quantifies the overall difference between the original and processed images, normalized by the magnitude of the original image's pixel values. This metric is useful for assessing the general accuracy of image reconstruction.

$$\text{Re} = \frac{\sum_{i=1}^{M} \sum_{j=1}^{N} |I(i,j) - K(i,j)|}{\sum_{i=1}^{M} \sum_{j=1}^{N} |I(i,j)|} \tag{7}$$

Lower values of Re indicate better similarity between the images.

E. Structural Similarity Index (SSIM)

The Structural Similarity Index (SSIM) measures the perceptual difference between two images. Unlike MSE-based metrics, SSIM considers changes in structural information, luminance, and contrast, making it more aligned with human visual perception. SSIM ranges from -1 to 1, with 1 indicating identical images.

$$\text{SSIM}(I, K) = \frac{(2\mu_I \mu_K + C_1)(2\sigma_{IK} + C_2)}{(\mu_I^2 + \mu_K^2 + C_1)(\sigma_I^2 + \sigma_K^2 + C_2)} \tag{8}$$

In this formula, μ_I and μ_K are the mean values, σ_I^2 and σ_K^2 are the variances, and σ_{IK} is the covariance of the original and processed images. The constants C_1 and C_2 are used to stabilize the formula when the denominator is close to zero, typically defined as:

$$C_1 = (k_1 L)^2, \quad C_2 = (k_2 L)^2 \tag{9}$$

where L is the dynamic range of the pixel values (e.g., 255 for 8-bit images), and k_1 and k_2 are small constants (e.g., $k_1 = 0.01$, $k_2 = 0.03$).

V. Conclusion

The conclusion goes here.

Acknowledgment

The authors would like to thank...

TABLE III: PSNR Error Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	26.33	28.50	25.72	46.74	30.84	19.00	27.66
benchmark_custom	24.30	26.10	24.80	43.41	30.38	17.42	25.88
benchmark	27.83	29.83	26.33	44.62	33.84	18.61	30.73
bicubic	27.32	29.51	25.91	44.54	33.46	17.96	30.27
lanczos	27.26	29.49	25.85	44.53	33.36	17.85	30.18
linear	27.83	29.83	26.33	44.62	33.84	18.61	30.73
nearest	26.45	28.73	25.55	44.29	32.36	17.74	29.02
proposed_algorithm	25.89	28.16	26.10	44.65	32.47	17.51	27.82

TABLE IV: NAE Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	0.0518	0.0461	0.0708	0.0134	0.0186	0.1077	0.0455
benchmark_custom	0.0789	0.0728	0.0867	0.0212	0.0259	0.1370	0.0680
benchmark	0.0533	0.0489	0.0727	0.0185	0.0179	0.1208	0.0404
bicubic	0.0565	0.0505	0.0772	0.0186	0.0195	0.1312	0.0438
lanczos	0.0573	0.0512	0.0782	0.0186	0.0204	0.1337	0.0453
linear	0.0533	0.0489	0.0727	0.0185	0.0179	0.1208	0.0404
nearest	0.0584	0.0505	0.0776	0.0188	0.0189	0.1312	0.0470
proposed_algorithm	0.0621	0.0548	0.0758	0.0177	0.0194	0.1339	0.0530

TABLE V: CC Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	0.9882	0.9866	0.9786	0.9998	0.9916	0.9542	0.9873
benchmark_custom	0.9811	0.9768	0.9735	0.9997	0.9907	0.9339	0.9808
benchmark	0.9916	0.9901	0.9813	0.9997	0.9958	0.9492	0.9937
bicubic	0.9906	0.9894	0.9795	0.9997	0.9954	0.9414	0.9930
lanczos	0.9904	0.9893	0.9792	0.9997	0.9953	0.9399	0.9929
linear	0.9916	0.9901	0.9813	0.9997	0.9958	0.9492	0.9937
nearest	0.9885	0.9873	0.9777	0.9997	0.9941	0.9385	0.9907
proposed_algorithm	0.9869	0.9855	0.9804	0.9997	0.9943	0.9354	0.9877

TABLE VI: MSE Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	151.39	91.94	174.41	1.38	53.63	818.37	111.37
benchmark_custom	241.41	159.73	215.49	2.97	59.56	1177.32	168.09
benchmark	107.27	67.63	151.36	2.24	26.86	895.08	55.02
bicubic	120.48	72.81	166.61	2.28	29.31	1041.21	61.04
lanczos	122.28	73.18	169.12	2.29	30.01	1067.74	62.34
linear	107.27	67.63	151.36	2.24	26.86	895.08	55.02
nearest	147.27	87.07	180.97	2.42	37.75	1095.27	81.52
proposed_algorithm	167.41	99.41	159.55	2.23	36.80	1153.37	107.45

TABLE VII: RE Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	0.0518	0.0461	0.0708	0.0134	0.0186	0.1077	0.0455
benchmark_custom	0.0789	0.0728	0.0867	0.0212	0.0259	0.1370	0.0680
benchmark	0.0533	0.0489	0.0727	0.0185	0.0179	0.1208	0.0404
bicubic	0.0565	0.0505	0.0772	0.0186	0.0195	0.1312	0.0438
lanczos	0.0573	0.0512	0.0782	0.0186	0.0204	0.1337	0.0453
linear	0.0533	0.0489	0.0727	0.0185	0.0179	0.1208	0.0404
nearest	0.0584	0.0505	0.0776	0.0188	0.0189	0.1312	0.0470
proposed_algorithm	0.0621	0.0548	0.0758	0.0177	0.0194	0.1339	0.0530

REFERENCES

[1] B. Gaster, L. Howes, D. R. Kaeli, P. Mistry, and D. Schaa, *Heterogeneous computing with openCL: revised openCL 1.* Newnes, 2012.

[2] A. H. Ashtari, M. J. Nordin, and S. M. M. Kahaki, "Double line image rotation," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3370–3385, 2015.

[3] W. Park, G. Leibon, D. N. Rockmore, and G. S. Chirikjian, "Accurate image rotation using hermite expansions," *IEEE Transactions on Image Processing*, vol. 18, no. 9, pp. 1988–2003, 2009.

[4] H.-C. Kwon, H.-J. Cho, and H.-Y. Kwon, "A study on high speed image rotation algorithm using cuda," *The Journal of the Institute of Internet, Broadcasting and Communication*, vol. 16, no. 5, pp. 1–6, 2016.

[5] P. R. Evans, "Rotations and rotation matrices," *Acta Crystallographica Section D: Biological Crystallography*, vol. 57, no. 10, pp. 1355–1359, 2001.

[6] J. Wang and J. Watada, "Panoramic image mosaic based on surf algorithm using opencv," in *Proceedings of the IEEE*, pp. 1–7, 2021.

TABLE VIII: SSIM Analysis of Various Sample Images

Methods	Images						
	astronaut	**brick**	**chelsea**	**color**	**moon**	**page**	**text**
affine_transform	0.9352	0.9354	0.8363	0.9929	0.9555	0.7894	0.9136
benchmark_custom	0.8812	0.8794	0.7790	0.9893	0.9276	0.6919	0.8428
benchmark	0.9390	0.9402	0.8222	0.9921	0.9619	0.7269	0.9273
bicubic	0.9335	0.9367	0.8088	0.9917	0.9574	0.7095	0.9174
lanczos	0.9317	0.9354	0.8059	0.9916	0.9557	0.7030	0.9134
linear	0.9390	0.9402	0.8222	0.9921	0.9619	0.7269	0.9273
nearest	0.9218	0.9281	0.8040	0.9904	0.9507	0.7079	0.9014
proposed_algorithm	0.9148	0.9185	0.8095	0.9902	0.9483	0.7090	0.8837